

NUS Orbital 2026



Milestone 01 Artemis Resubmission

[Poster](#) [Video Walkthrough](#) [Deployed App](#) [GitHub Logs](#)

Atlas (Team ID:)

Wang Qichen (A0337219B)

Feng Yi (A0322277E)

Proposed Level of Achievement: Artemis

Contents

Resubmission Response	3
Deployment and Submitted Materials	3
Motivation	4
Aim and Artemis Scope	4
User Stories	5
Feature Ideation	7
Overall Architecture	8
Software Engineering At Project Level	9
Project Structure	9
Project Management	9
Implemented Core Features	10
Authentication and Protected Access	10
Campus Dashboard and Assistant	10
Campus Map Rendering	10
Implemented Supporting Features	11
Location Search and Route Interaction	11
NUS Bus Context	11
Mobile and iOS Experience	11
Testing Strategy	12
Automated Checks	13
Manual Regression Checks	13
CI/CD Pipeline	14
UML Diagrams	15
Authentication Flow	15
Map Search and Routing Flow	15
Video Walkthrough	16
Milestone 01 Progress	16
Planned Next Steps	16

Resubmission Response

This report has been rewritten for Artemis Milestone 01 re-evaluation. The previous submission did not clearly demonstrate Artemis-level planning, project-level software engineering, automated testing, CI/CD, and walkthrough evidence. This version makes those items explicit and connects them to the current codebase, deployed application, and GitHub workflow.

No.	Expectation	Where Addressed In This Report
1	20–23 user stories	The User Stories section contains 22 stories grouped by student, route, facility, schedule, accessibility, administrator, and engineering personas.
2	Ideation for all ≥ 9 features	The Feature Ideation section describes 10 planned or implemented features, each with problem, idea, Milestone 01 status, and later extension.
3	Project-level software engineering	The Software Engineering At Project Level section documents GitHub issues, PRs, module boundaries, branch policy, dependency management, environment management, and risk handling.
4	Artemis M1 page depth	The report has been expanded beyond the previous Apollo-like summary and is compiled as a full Artemis M1 technical report.
5	Automated testing documentation	The Testing Strategy section documents automated checks, manual regression tests, and current test limitations.
6	CI/CD setup and documentation	A GitHub Actions workflow has been added at <code>.github/workflows/ci.yml</code> ; the CI/CD section documents frontend build, backend tests, Docker build, and deployment path.
7	Video walkthrough	The Video Walkthrough section states what the submitted video demonstrates and links it from the cover page.

Deployment and Submitted Materials

The deployed web application is available at: orbital-artemis-armap-nus.onrender.com.

The GitHub repository and issue tracker are available at: [Atlas GitHub repository](#).

The submitted video walkthrough is linked on the cover page. It is intended to show the app as a working prototype, not just a code repository: login, protected access, campus dashboard, map rendering, mobile layout, and the current user flow are demonstrated in sequence.

Motivation

Navigating NUS is difficult for new students, exchange students, visitors, and even current students moving between unfamiliar faculties. Public maps may show buildings, but students often need more than a building polygon: they need to know which entrance to use, whether the building has facilities they need, how long it may take to move there, and whether their next class or meeting changes the priority of route suggestions.

Atlas is motivated by the gap between static campus maps and the real decision-making students perform every day. A student does not simply ask “where is COM2?” The more realistic question is: “I am currently at one location, I have class soon, I may need a bus, and I need to know the most useful next action.” Atlas therefore combines authentication, campus map rendering, facility discovery, schedule awareness, bus context, and a daily assistant interface.

Aim and Artemis Scope

Atlas aims to become a campus assistant for NUS students that supports map-based navigation, facility discovery, schedule-aware guidance, and future AR-style indoor navigation. For Artemis Milestone 01, our goal is not only to produce a working prototype, but also to establish a project foundation that can scale into later milestones.

For this milestone, we focus on:

- a deployed full-stack application using React, Go, SQLite, and Docker;
- authentication with email/password, demo mode, protected routes, JWTs, and Google Sign-In;
- campus data APIs for buildings, facilities, schedules, recommendations, sync status, and bus context;
- MapLibre/OpenStreetMap rendering and early route-oriented interactions;
- mobile usability and an iOS app shell;
- project-level engineering practices: GitHub issues, branches, pull requests, CI, build checks, and deployment documentation.

User Stories

The 22 user stories below define the wider Artemis product vision. Milestone 01 implements the first layer of this vision and creates a platform for later AR, indoor mapping, and personalization work.

1. As a new NUS student, I want to search for a campus building by name so that I can find unfamiliar locations quickly.
2. As a student rushing to class, I want route guidance from my current location to a target building so that I can arrive on time.
3. As a student with back-to-back lessons, I want Atlas to consider my schedule so that it can highlight the next relevant destination.
4. As a student inside an unfamiliar building, I want to find facilities such as restrooms, lifts, study spaces, and printers so that I can solve immediate needs.
5. As a student with accessibility needs, I want lift and accessible-route information so that I can avoid unsuitable paths.
6. As an exchange student, I want building and facility names to be searchable without knowing exact abbreviations so that I can navigate NUS confidently.
7. As a student using a phone outdoors, I want the interface to be readable on mobile so that I can use the app while walking.
8. As a student with limited mobile data, I want the app to load essential map and schedule information efficiently so that navigation remains practical.
9. As a student planning my day, I want to add and remove schedule items so that Atlas can reflect my actual timetable.
10. As a student leaving a lecture, I want recommendations for nearby useful resources so that I can decide what to do next.
11. As a student using NUS shuttle buses, I want bus stop and arrival context so that I can decide whether to walk or take a bus.
12. As a student who values privacy, I want login-protected access so that personal schedule data is not exposed publicly.
13. As a returning user, I want a persistent account flow so that I do not need to re-enter setup information every time.
14. As a user who wants to try the project quickly, I want demo mode so that I can evaluate Atlas without creating a full account.
15. As a user with a Google account, I want Google Sign-In so that login is faster and closer to real student authentication patterns.
16. As a project evaluator, I want a deployed app URL so that I can inspect the prototype without local setup.
17. As a project evaluator, I want a video walkthrough so that I can understand feature usability without guessing from screenshots.
18. As a developer, I want frontend and backend modules to be separated so that map, auth, and data work can progress independently.
19. As a developer, I want CI checks on pull requests so that broken builds and backend regressions are caught before merge.

20. As a developer, I want environment variables documented so that local and production setups are reproducible.
21. As a maintainer, I want seeded demo data so that the application can be tested consistently.
22. As a future Artemis team member, I want a foundation for AR and indoor navigation so that later milestones can build on working map, auth, and data layers.

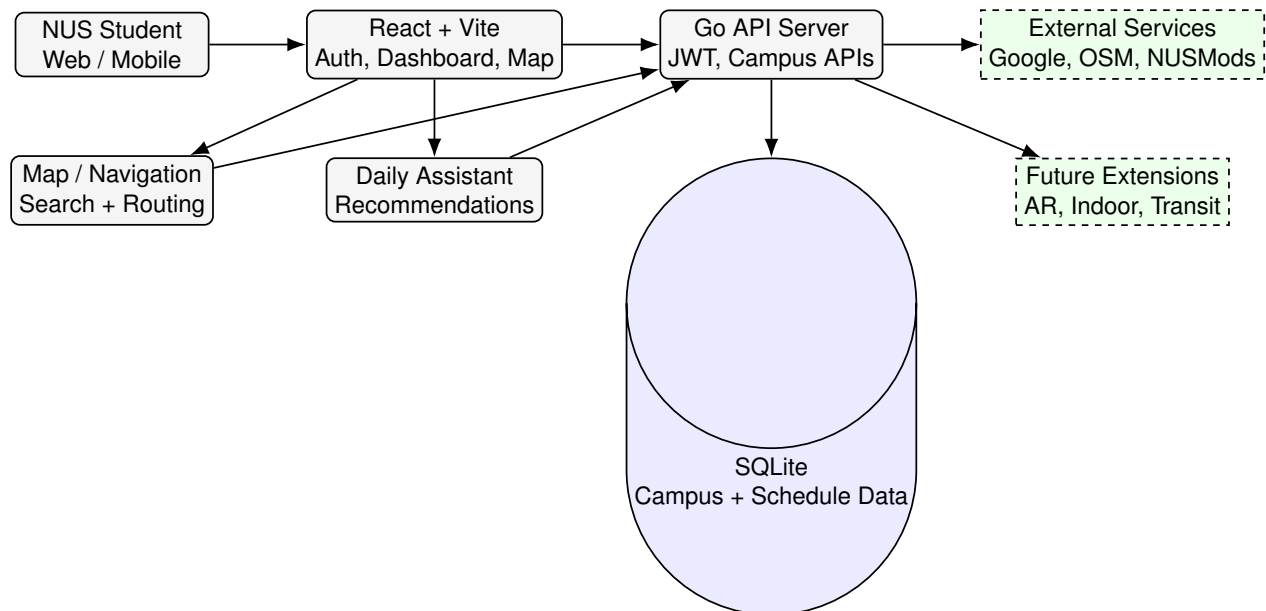
Feature Ideation

The table below records ideation for 10 features. Each feature is treated as part of the product plan, not only as an isolated implementation task.

No.	Feature	Ideation and User Value	M1 Status and Later Extension
1	Repository foundation	Create a reproducible full-stack project with clear ownership between frontend, backend, deployment, and documentation.	Implemented. Later work adds stricter PR templates and release tags.
2	Email and demo login	Let students and evaluators enter the app through either real credentials or a low-friction demo path.	Implemented with JWT flow. Later work adds password reset and stronger account management.
3	Google Sign-In	Support a realistic student login path using a trusted identity provider.	Implemented through frontend ID token submission and backend verification. Later work can connect to NUS identity if available.
4	Protected dashboard	Keep schedule and personalized recommendations behind an authentication boundary.	Implemented with protected frontend routes and backend authorization headers.
5	Campus data dashboard	Present buildings, facilities, schedules, recommendations, and sync status in a student-facing workspace.	Implemented with seeded SQLite data and API endpoints. Later work expands data coverage.
6	Map rendering	Provide an interactive campus map instead of static screenshots.	Implemented with MapLibre and OSM-style map rendering. Later work adds richer indoor overlays.
7	Search and route tracing	Allow users to move from typed destination intent to route-oriented map interaction.	Partially implemented through search, markers, and route drawing. Later work improves routing accuracy.
8	NUS bus context	Include campus transit as part of navigation decisions.	Implemented as bus stops, routes, arrivals, active vehicles, alerts, and context endpoints with demo fallback.
9	Mobile and iOS shell	Make the app usable in the realistic mobile setting where navigation happens.	Implemented responsive UI and Capacitor iOS shell. Later work can add native plugins.
10	CI/CD and deployment	Make engineering quality visible through automated checks and reproducible deployment.	CI added through GitHub Actions; deployment documented for Render and Docker/Caddy.

Overall Architecture

The architecture follows a modular system design: user interface, authentication, campus assistant/dashboard, map/navigation, backend APIs, persistent data, and external services. This is important at project level because each module can evolve without rewriting the whole system.



Atlas overall architecture

Software Engineering At Project Level

Project Structure

```
Orbital_Artemis_ArMap_Nus/
|-- frontend/ React + Vite frontend and mobile app shell
| |-- src/ Auth, dashboard, campus map, and UI code
| '-- ios/ Capacitor iOS project
|-- backend/ Go API server
| |-- cmd/server/ Server entry point
| '-- internal/atlas/ Auth, campus data, routing, and API logic
|-- deploy/ Caddy and production deployment notes
|-- .github/workflows/ CI workflow for build, test, and Docker image
|-- Dockerfile Production image definition
|-- docker-compose.yml Local production-like stack
'-- readme.tex Milestone report source
```

Project structure

Project Management

The team uses GitHub issues as the task log, feature branches for isolated work, and pull requests for integration. Issues are used not only as a checklist, but also as a planning surface: authentication, map rendering, search, dashboard integration, mobile usability, bus integration, and deployment are separated into trackable units. This prevents the project from becoming a single unreviewable prototype.

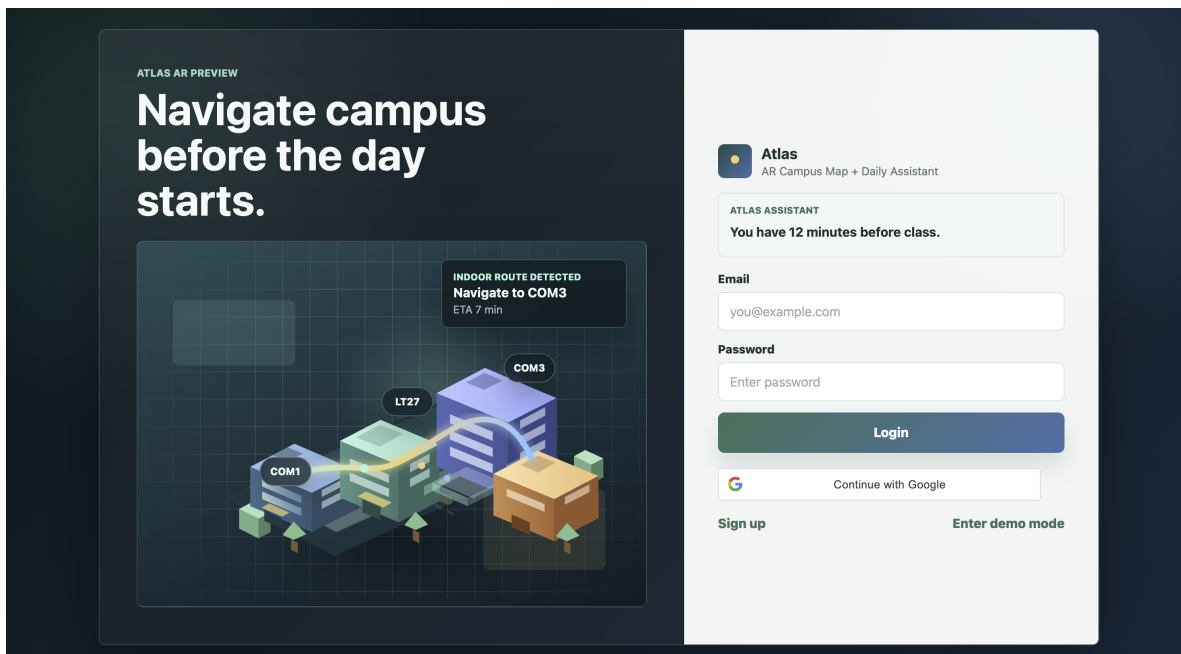
At project level, Atlas applies these principles:

- **Separation of concerns:** React owns user interaction; Go owns authentication, data APIs, and trusted external calls; SQLite owns seeded persistent state.
- **Reproducibility:** README, Dockerfile, Docker Compose, and environment variables document how to run the same app locally or in production.
- **Incremental delivery:** Milestone 01 establishes auth, dashboard, map, bus, deployment, and mobile shell before attempting heavier AR features.
- **Risk reduction:** uncertain future work such as indoor AR is postponed until map, data, and identity foundations are reliable.
- **Reviewability:** feature branches and PRs keep map changes, auth changes, and deployment changes inspectable.

Implemented Core Features

Authentication and Protected Access

Authentication allows students to enter Atlas through email/password login, registration, demo mode, or Google Sign-In. Once authenticated, users can access protected pages such as the dashboard and map.



Authentication Screen

On the backend, Atlas verifies credentials and issues a JWT. For Google Sign-In, the frontend obtains a Google ID token and sends it to the backend, where it is verified against the configured Google client ID. The backend then returns the same application JWT format used by email login, keeping the rest of the application independent of the authentication method.

Campus Dashboard and Assistant

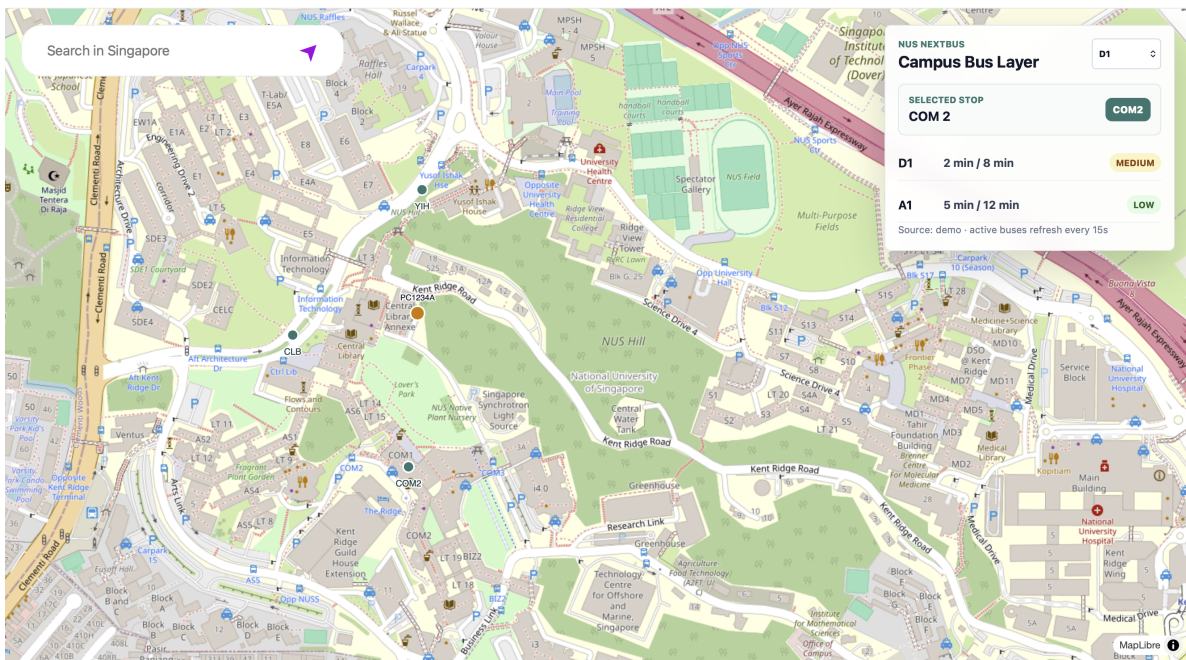
The dashboard connects campus data to the student's daily context. Instead of showing raw database rows, Atlas groups buildings, facilities, schedules, recommendations, NUSMods sync status, and bus information into sections that can be scanned quickly.

This feature is important for Artemis because it moves the project beyond a simple map. The assistant layer is where later schedule-aware navigation, reminders, and AR hints can be connected.

Campus Map Rendering

The map feature gives users an interactive visual interface for campus navigation. Atlas renders a MapLibre/OpenStreetMap-based campus map centered around NUS and prepares the surface for future indoor overlays and AR markers.

The map logic is separated into a map initialization module, a MapEngine helper, routing services, and UI components. This modular approach prevents the main map page from becoming a large untestable component.



Campus Map Screen

Implemented Supporting Features

Location Search and Route Interaction

Location search improves usability by allowing users to type a destination instead of manually inspecting the map. Selecting a suggestion places a marker and moves the map to the chosen location. Route tracing then allows selected start and end points to be drawn on the map as an early prototype of the navigation goal.

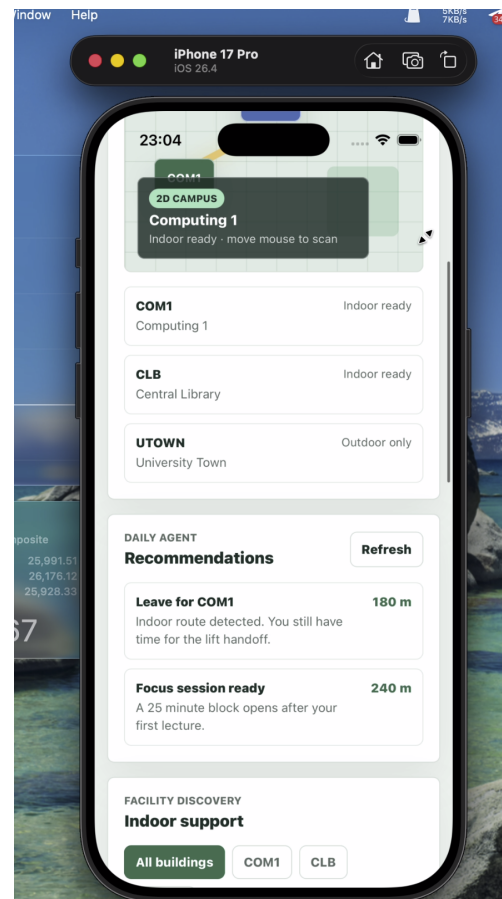
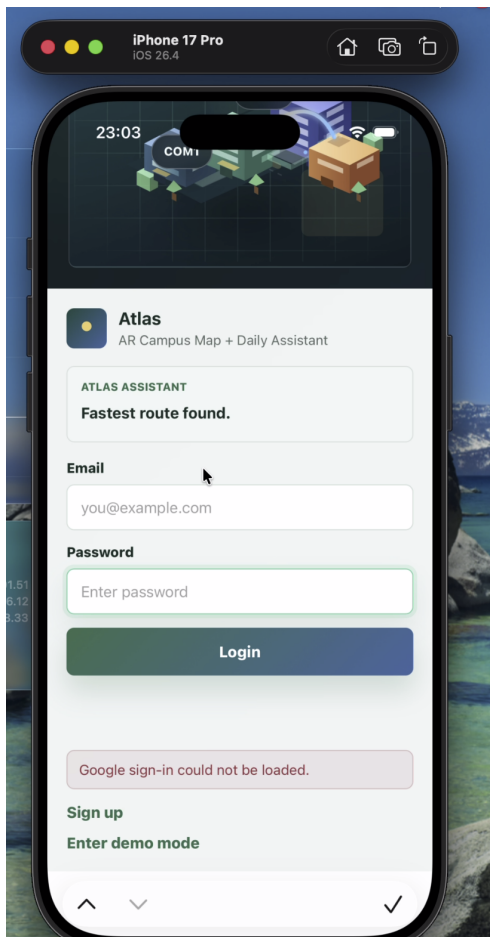
The state is split into display text and confirmed place objects. This simple model-view separation avoids treating incomplete text as a real geospatial endpoint.

NUS Bus Context

Atlas includes NUS bus endpoints for stops, routes, arrivals, active vehicles, alerts, and location-based bus context. When live API credentials are not configured, the backend can serve demo data. This design keeps local development usable while still preparing for real campus transit integration.

Mobile and iOS Experience

Campus navigation is often used while students are moving, so mobile usability is not optional. The frontend is designed to remain readable on common mobile widths, and a Capacitor iOS shell under `frontend/ios` loads the deployed HTTPS app.



Mobile app screens

Testing Strategy

Testing is documented as both automated checks and manual regression checks. The current codebase is still a Milestone 01 prototype, so the automated layer focuses on build and backend correctness gates, while manual checks cover user-facing workflows that are difficult to validate without a browser session.

Automated Checks

Check	Command / Location	Purpose
Frontend production build	<code>cd frontend && npm run build</code>	Confirms that the React/Vite app bundles successfully and catches broken imports or invalid build-time code.
Backend Go test gate	<code>cd backend && go test ./...</code>	Compiles all backend packages and runs Go tests as they are added. This catches API, auth, store, and scheduler regressions at package level.
Docker production image	<code>docker build -t atlas-ci .</code>	Confirms the production image can be built from the repository without relying on local developer state.
CI workflow	GitHub Actions	Runs the frontend build, backend tests, and Docker build from <code>ci.yml</code> on push and pull requests to <code>main</code> .

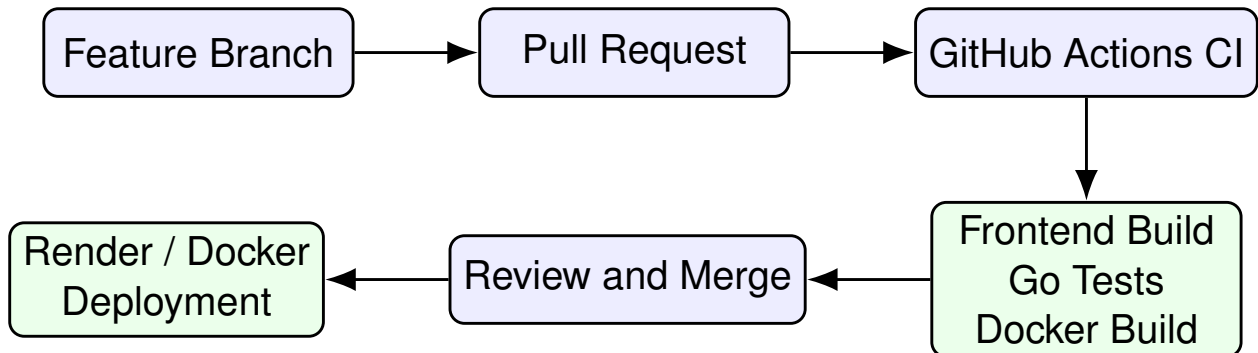
Manual Regression Checks

Workflow	Manner of Testing
Login and registration	Test demo login, email login, registration, invalid credentials, and protected-route redirection.
Google Sign-In	Test that the frontend obtains an ID token and the backend exchanges it for an app JWT.
Dashboard data	Verify buildings, facilities, schedule cards, recommendations, and sync status load after authentication.
Map rendering	Open the campus map and verify map tiles, NUS centering, markers, and route drawing behavior.
Bus context	Verify stops, routes, arrival cards, active vehicles, and demo fallback behavior when live credentials are absent.
Mobile layout	Use narrow browser viewport and iOS shell to confirm content does not overflow and main actions remain reachable.
Deployment smoke test	Open the Render deployment and check that frontend routes and backend APIs are served together.

Known limitation: Milestone 01 currently has stronger automated build gates than fine-grained unit tests. In the next milestone, the team will add focused tests for authentication handlers, schedule CRUD, recommendation logic, bus fallback behavior, and frontend interaction flows.

CI/CD Pipeline

A CI workflow has been added to the repository at `.github/workflows/ci.yml`. It runs on push and pull requests targeting `main`. This directly supports Artemis expectations because code quality is checked at project level rather than only through local manual testing.



Atlas CI/CD pipeline

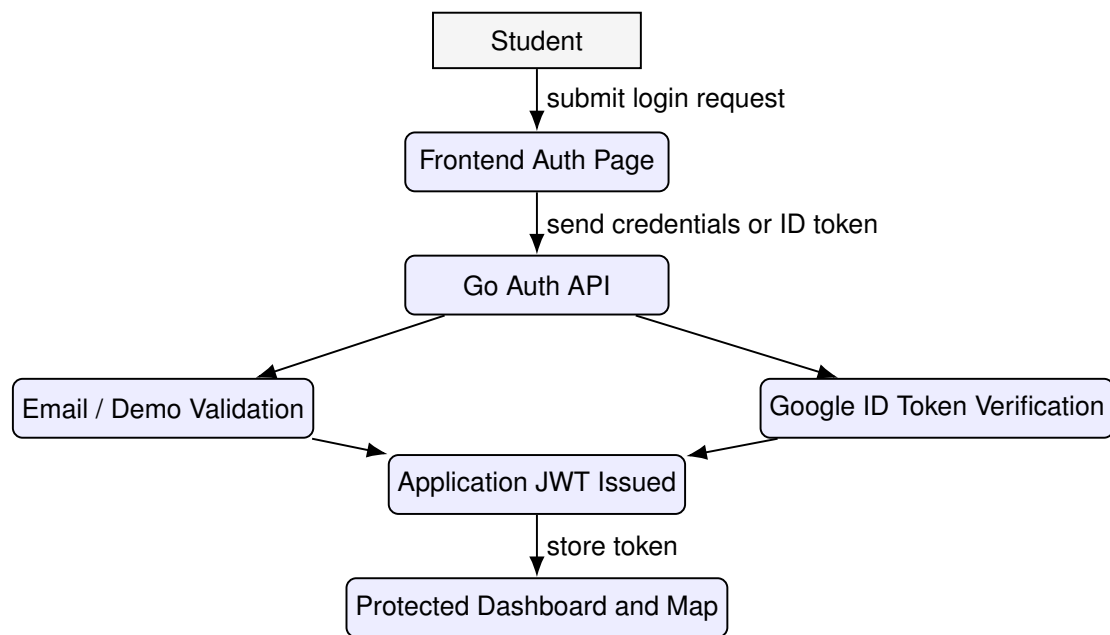
The CI jobs are:

- **Frontend build:** installs dependencies in `frontend/` with `npm ci` and runs `npm run build`.
- **Backend tests:** uses the Go version from `backend/go.mod`, downloads modules, and runs `go test ./...`
- **Docker build:** builds the production image after frontend and backend jobs pass.

The deployment path is documented through the Render-hosted app and the Docker/Caddy production stack. The intended release discipline is: work on feature branch, open PR, pass CI, merge to `main`, then deploy from the stable branch.

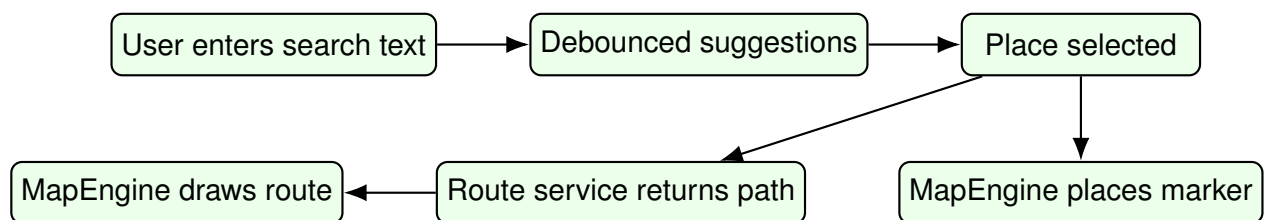
UML Diagrams

Authentication Flow



Authentication sequence for protected access

Map Search and Routing Flow



Map search and route drawing workflow

Video Walkthrough

The video link is included on the cover page as “Video Walkthrough.” The walkthrough is structured to demonstrate usability instead of only showing code. It covers:

- opening the deployed app from the public URL;
- entering through login or demo mode;
- reaching protected dashboard content after authentication;
- viewing campus data, schedule context, recommendations, and sync status;
- opening the campus map and demonstrating map-centered navigation behavior;
- showing mobile responsiveness through current mobile screenshots or viewport demonstration;
- explaining how the current prototype prepares for later AR and indoor navigation milestones.

This directly addresses the earlier weakness that the submission did not sufficiently demonstrate current feature usability.

Milestone 01 Progress

Milestone 01 now provides a working React frontend, Go backend, SQLite data layer, authentication flow, campus dashboard, map page, bus endpoints, mobile shell, deployment path, and CI workflow. The project is still early, but it is no longer only an ideation artifact: it is a deployed, inspectable, and build-checked prototype.

Planned Next Steps

- Add backend unit tests for login, registration, schedule CRUD, recommendation generation, and bus demo fallback.
- Add frontend interaction tests for login, protected routing, dashboard rendering, and map controls.
- Expand campus data coverage for rooms, entrances, lifts, accessible routes, and indoor facilities.
- Improve route accuracy with multimodal walking and shuttle-bus routing.
- Turn the iOS shell into a more native mobile experience if required by later milestones.
- Continue user validation with NUS students and refine the assistant workflow from feedback.
- Prepare AR-style indoor navigation only after stable data, routing, and mobile foundations are in place.